

Q1. Intelligent Customer Feedback Analysis

CODE

```
positive_words = ["good","excellent","great","awesome","happy","love","best"]
negative_words = ["bad","poor","worst","hate","terrible","disappointed","slow"]
def preprocess(text):
    text = text.lower().strip()
    for ch in ["."," ", "!", "?", ";", ":", "'", '"', "(", ")", "-"]:
        text = text.replace(ch, "")
    return text
def analyse_feedback(text):
    words = text.split()
    positive_count = sum(1 for w in words if w in positive_words)
    negative_count = sum(1 for w in words if w in negative_words)

    if positive_count > negative_count:
        return positive_count, negative_count, "Positive"
    elif negative_count > positive_count:
        return positive_count, negative_count, "Negative"
    return positive_count, negative_count, "Neutral"

n = int(input())

for i in range(n):
    feedback = input()
    clean_text = preprocess(feedback)
    positive_count, negative_count,result = analyse_feedback(clean_text)

    print(f"Feedback {i+1}")
    print("Preprocessed Feedback:",clean_text)
    print("Positive Keywords:",positive_count)
    print("Negative Keywords:",negative_count)
    print("Sentiment      :",result)
```

INPUT

2
The product is very good and excellent!
This is the worst hate it poor service.

OUTPUT

(no output)

Q2. Comprehensive Password Strength Checker

CODE

```
def check_strength(p):
    if len(p) < 6:
        return "Weak"
    u = any(c.isupper() for c in p)
    l = any(c.islower() for c in p)
    d = any(c.isdigit() for c in p)
    s = any(not c.isalnum() for c in p)
    score = u + l + d + s

    if score == 4 and len(p) ≥ 10:
        return "Very Strong"
    elif score ≥ 3 and len(p) ≥ 8:
        return "Strong"
    elif score ≥ 2:
        return "Moderate"
    else:
        return "Weak"
password = input()
print("Password Strength:", check_strength(password))
```

INPUT

Admin@12345

OUTPUT

(no output)

Q3. Structured Server Log Error Analyser

CODE

```
def analyze_logs():
    seen = set()
    counts = {"Database Error":0, "Timeout Error": 0, "Auth Error": 0, "Other Error": 0}

    while True:
        try:
            line = input().strip()
            if not line: break

            if "Error" in line.upper() and line not in seen:
                seen.add(line)
                print("Extracted:", line)

                up = line.upper()
                if "DB" in up or "DATA" in up: counts["Database Error"] += 1
                elif "TIME" in up: counts["Timeout Error"] += 1
                elif "AUTH" in up or "LOG" in up: counts["Auth Error"] += 1
                else : counts["Other Error"] += 1
        except EOFError:
            break

    print("\n--- Error Summary ---")
    for k, v in counts.items():
        print(f"{k}: {v}")
analyze_logs()
```

INPUT

ERROR: DB connection failed
ERROR: DB connection failed
INFO: Server running normally
ERROR: Connection TIMEOUT
ERROR: AUTH login failed

OUTPUT

(no output)

Q4. Email Domain Frequency Analyser

CODE

```
def check_emails():
    f = {}
    while True:
        try:
            em = input().strip()
            if not em: break
            if "@" in em and em.count("@") == 1 and "." in em.split("@")[1]:
                dom = em.split("@")[1].strip()
                f[dom] = f.get(dom, 0) + 1
        except EOFError:
            break
    print("\n--- Domain Frequency ---")
    mx_d, mx_c = "None", 0
    for d, c in sorted(f.items()):
        print(f"{d}: {c}")
        if c > mx_c: mx_d, mx_c = d, c
    print(f"\nMost Common Domain: {mx_d} ({mx_c}")

check_emails()
```

INPUT

test1@gmail.com
test2@yahoo.com
test3@gamil.com
invalidemail

OUTPUT

(no output)